JSP - Servlet

Chapitre 4: Externalisation du code Java

Dr. Ahmad Khoureich Ka

UADB

L3 D2AW

1. Introduction
2. JSP Expression Language (EL)
   - Accès aux propriétés d'un objet
   - Opérateurs
   - Objets implicites
3. Les JavaBeans
   - Un exemple de JavaBean
   - Déclaration d'un JavaBean
   - Initialisation de propriétés
   - Lecture d'une propriété
4. JSP Standard Tag Library (JSTL)
   - JSTL : core
   - JSTL : functions

Introduction

- Un des objectifs de la technologie JSP est le respect du modèle MVC.
- Les JSP sont essentiellement utilisés pour la présentation.
- Il est recommendé d'éviter au maximum l'utilisation d'élements de scripts dans une page JSP.
- L'externalisation du code Java facilite grandement l'écriture et la maintenance de la page JSP.

JSP Expression Language (EL)

- JSP EL permet en quelque sorte de remplacer l'élément de script `<%= expression %>` dans une page jsp.
- Syntaxe : `${expression}`
- `expression` peut être :
  - une variable : Exemple : `${prixUnitaire}`
  - une expression arithmétique. Exemple : `${2*(4+6)}`
  - une expression logique. Exemple : `${moyenne >= 10}`
  - la propriété d'un objet. Exemple : `${rectangle.longueur}`
  - l'attribut d'un objet implicite. Exemple : `${param.username}`

JSP EL : accès aux propriétés d'un objet

- Les variables accessibles avec EL sont les variables de scope, c'est à dire les attributs attachés à la page jsp (`pageContext`), à la requête (`request`), à la session (`session`) ou au contexte de l'application (`application`).
- Le parser EL recherche une variable donnée successivement parmis les attributs attachés à :
  1. la page jsp,
  2. la requête,
  3. la session,
  4. l'application.
- Remarque : Les variables définies dans une déclaration `<%! %>` ne sont pas disponible avec EL.

JSP EL : accès aux propriétés d'un objet

- Pour accéder à une propriété d'un objet avec EL, les opérateurs
  - ou `[]` peuvent être utilisés.

- Exemple :
  Classe `Cercle`

```
public class Cercle{

    private int rayon;

    public Cercle(int rayon) {
        this.rayon = rayon;
    }

    public int getRayon() {
        return rayon;
    }
}
```

Accès aux propriétés d'un objet

- Exemple : (suite)
  Dans un JSP :

```
<%
    Cercle cercle = new Cercle(12);
    request.setAttribute("cercle", cercle);

    List listAchat = new ArrayList();
    listAchat.add("lait");
    listAchat.add("savon");
    pageContext.setAttribute("listAchat", listAchat);

    pageContext.setAttribute("index", 1);
%>
```

Accès aux variables :

```
Rayon du cercle: ${cercle.rayon} <br/>
Rayon du cercle: ${cercle["rayon"]} <br/>
Article ${index + 1} de listAchat: ${listAchat[index]}
```

JSP EL : Opérateurs

- Liste des opérateurs reconnu par EL

Priorité	Opérateur
1	. , []
2	( )
3	signe -, ! ou not, empty
4	*, / ou div, % ou mod
5	+, -
6	< ou lt, > ou gt, <= ou le, >= ou ge,
7	== ou eq, != ou ne,
8	&& ou and
9	|| ou or,
10	?:

- L'opérateur **empty** permet de tester si une collection, un tableau ou une chaine de caractères est **null** ou vide.

JSP EL : Opérateurs

☞ Exemples :

- La variable `prixU` n'existe pas : `${empty prixU}` donne `true`
- Le calcul `${12 div 6}` donne `2.0`
- La comparaison `${12 lt 6}` donne `false`
- Soit `msg` un attribut de requête.
  Le code

```
${msg eq "ok" ? "bonjour Modou" : ""}
```

  est équivalent à

```
<%
  String msg = (String) request.getAttribute("msg");
  out.print((msg != null && msg.equals("ok")) ? "bonjour Modou" : "");
%>
```

JSP EL : Objets implicites

- **`initParam:`** retourne la valeur du paramètre d'initialisation de contexte spécifié.

```
${initParam.urlMysql}
```

- **`param:`** retourne la valeur du paramètre de requête monovalué spécifié.
  Exemple : Afficher la valeur du paramètre **`username`**

```
${param.username}
```

  est équivalent à

```
<%
   String username = request.getParameter("username");
   out.print(username == null ? "" : username);
%>
```

JSP EL : Objets implicites

- **`paramValues:`** retourne un tableau contenant l'ensemble des valeurs associées au paramètre de requête multi-valué spécifié.
  Exemple : **`livres`** est un paramètre multivalué

```
${paramValues.livres[0]}
${paramValues["livres"][1]}
```

- **`header:`** retourne la valeur de l'en-tête de requête monovalué spécifié.

```
${header["accept-language"]}
```

- **`headerValues:`** retourne un tableau contenant l'ensemble des valeurs associées à l'en-tête de requête multi-valué.

- **`cookie:`** pour accéder à un cookie.
  Exemple : Afficher la valeur du cookie **`pub`**

```
${cookie["pub"].value}
```

JSP EL : Objets implicites

Accès direct à un attribut de scope sans recherche

- **`pageScope:`** retourne la valeur de l'attribut de **`pageContext`** spécifié.
- **`requestScope:`** retourne la valeur de l'attribut de **`request`** spécifié.
- **`sessionScope:`** retourne la valeur de l'attribut de **`session`** spécifié.
  Exemple : Afficher l'objet **`panier`** attribut de session

```
${sessionScope.panier}
```

- **`applicationScope:`** retourne la valeur de l'attribut de **`application`** spécifié.

Les JavaBeans

- Un JavaBean est une classe Java dont la définition respecte un certain nombre de conventions.
- Ces spécifications sont les suivantes :
  - Implémenter l'interface `Serializable`.
  - Avoir un constructeur sans paramètre.
  - Toutes les propriétés doivent être accessibles à l'aide d'accesseurs publics `getter` et `setter`.
- Une propriété de JavaBean peut être en lecture/écriture, lecture seule ou écriture seule.
- Le JavaBean est utile pour éviter les éléments de scripts `<%! déclaration %>` et `<% scriptlet %>` dans un JSP.

Un exemple de JavaBean

```
package modele;
import java.io.Serializable;

public class Etudiant implements Serializable{
   private String nom;
   private float[] notes;
   private float moyenne;

   public String getNom() { return nom; }
   public void setNom(String nom) { this.nom = nom; }

   public float[] getNotes() { return notes; }
   public void setNotes(float[] notes) { this.notes = notes; }

   public float getMoyenne() {
      if(notes==null){ return -1f; }
      float total = 0f;
      for(float n : notes){ total += n; }
      return total / notes.length;
   }
}
```

Déclaration d'un JavaBean dans un jsp

- Avant d'utiliser un JavaBean dans une page jsp, celui-ci doit être déclaré avec l'action jsp **`<jsp:useBean>`**.
- Syntaxe :

```
<jsp:useBean id="nom" class="classeDuJavabean"
    scope="page|request|session|application" />
```

  - **`id`** : spécifie le nom de l'objet JavaBean à créer ou à accéder.
  - **`class`** : spécifie le nom complet de la classe du JavaBean.
  - **`scope`** : spécifie la portée du JavaBean (page, request, session ou application), sa valeur par défaut est **`page`**.

Déclaration d'un JavaBean

- Exemple :

```
<jsp:useBean id="etudiant" class="modele.Etudiant"
    scope="request"/>
```

- Remarques :
  - Lorsque l'objet JavaBean **etudiant** est dans le scope (ici la requête), l'instruction de l'exemple permet d'y accéder.
  - Lorsque l'objet **etudiant** n'existe pas dans la requête, l'instruction de l'exemple permet de créer un objet de la classe **modele.Etudiant** nommé **etudiant** puis de le définir comme attribut de requête.
  - L'objet JavaBean déclaré avec **<jsp:useBean/>** est disponible pour les éléments de script (expression et scriptlet) et pour EL.

JavaBean : Initialisation de propriétés

Initialisation à l'aide de l'action `<jsp:setProperty>`

- Syntaxes :
  - Initialiser une propriété du JavaBean avec une valeur donnée.

```
<jsp:setProperty name="nomDuBean" property="propriété"
    value="valeur" />
```

  - Initialiser une propriété du JavaBean avec un paramètre de requête.

```
<jsp:setProperty name="nomDuBean" property="propriété"
    param="paramRequete" />
```

    Lorsque la propriété a le même nom que le paramètre l'attribut `param` peut être omis.

  - Initialiser toutes les propriétés du JavaBean avec les valeurs des paramètres de requête portant les mêmes noms.

```
<jsp:setProperty name="nomDuBean" property="*" />
```

JavaBeans : Initialisation de propriétés

Exemple 1 : Initialiser à la création du JavaBean **etudiant** la propriété **nom**

```
<jsp:useBean id="etudiant" class="modele.Etudiant" scope="request">
   <jsp:setProperty name="etudiant" property="nom" value="modou"/>
</jsp:useBean>
```

Exemple 2 : Réinitialiser la propriétés **nom** du JavaBean **etudiant**

```
1 <jsp:useBean id="etudiant" class="modele.Etudiant" scope="request"/>
2 <jsp:setProperty name="etudiant" property="nom" value="Lamine"/>
```

- Remarque :
  L'instruction 1 avant la 2 est importante car elle permet de créer l'objet **etudiant** au cas où celui-ci n'existe pas avant d'initialiser sa propriété **nom**.

JavaBeans : Initialisation de propriétés

Exemple 3 : soit le formulaire :

```
Nom: <input type="text" name="nom" value="" /><br/>
note algo: <input type="text" name="notes" value="" /><br/>
note math: <input type="text" name="notes" value="" /><br/>
```

- Réinitialiser la propriété **notes** du JavaBean **etudiant** avec le paramètre de requête multivaluée **notes**

```
<jsp:useBean id="etudiant" class="modele.Etudiant" scope="request"/>
<jsp:setProperty name="etudiant" property="notes"/>
```

- Initialiser d'un seul coup à la création du JavaBean **etudiant** toutes ses propriétés avec les paramètres de requête

```
<jsp:useBean id="etudiant" class="modele.Etudiant" scope="request">
   <jsp:setProperty name="etudiant" property="*"/>
</jsp:useBean>
```

JavaBeans : Lecture d'une propriété

Lecture à l'aide de l'action `<jsp:getProperty>`

- Syntaxe :

```
<jsp:getProperty name="nomDuBean" property="propriété" />
```

  - **`name`** : spécifie le nom de l'objet JavaBean.
  - **`property`** : spécifie le nom de la propriété à lire.

- Exemple :

```
<jsp:useBean id="etudiant" class="modele.Etudiant" scope="request"/>
Nom étudiant: <jsp:getProperty name="etudiant" property="nom"/>
```

Lecture d'une propriété

☞ Remarques :

- Une propriété de JavaBean peut être lue à l'aide d'un élément de script.

```
<jsp:useBean id="etudiant" class="modele.Etudiant" scope="request"/>
Moyenne étudiant: <%= etudiant.getMoyenne() %>
```

- Une propriété de JavaBean peut être lue à l'aide de JSP EL. Lorsque la valeur est `null` EL n'affiche rien.

```
Moyenne étudiant: ${requestScope.etudiant.moyenne}
```

JSTL : JSP Standard Tag Library

- JSTL est un ensemble de librairies de tags standardisés permettant de remplacer certains traitements qui jusque là n'étaient possibles qu'avec les scriptlets.
- JSTL regroupe 5 librairies de tags qui sont [1] :
  - **`core:`** librairies de base destinée entre autre à l'utilisation des variables, des structures de contrôle, à la gestion des urls.
  - **`functions:`** pour la lecture de la taille d'une collection et la manipulation de chaine.
  - **`fmt:`** pour l'internationnalisation (langue et formatage spécifique au client).
  - **`XML:`** pour l'utilisation des documents XML.
  - **`sql:`** pour l'accès aux bases de données.

1. Dans ce chapitre nous nous limiterons aux librairies core et functions.

Installation de la librairie JSTL

- Inclure dans **pom.xml** la dépendance :

```
<dependency>
   <groupId>javax.servlet</groupId>
   <artifactId>jstl</artifactId>
   <version>1.2</version>
</dependency>
```

JSTL : core

- JSTL core est la librairie de base. Elle regroupe les tags pour :
  - l'utilisation de variables : **`set`** et **`remove`**.
  - les structures de contrôle : **`if`**, **`choose`**, **`forEach`** et **`forTokens`**.
  - la manipulation des urls : **`url`**, **`redirect`** et **`import`**.
  - d'autres actions : **`out`** et **`catch`**.
- Pour utiliser la librairie JSTL core dans une page JSP, il suffit de la déclarer avec une directive **`<%@ taglib %>`**

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

JSTL : `<c:set/>`

- Le tag `<c:set/>` permet de définir une variable ainsi que sa portée (scope) ou la valeur d'une propriété d'un JavaBean ou d'un `Map`.
- Syntaxe
  - Définition d'une variable :

```
<c:set var="variable" value="valeur" scope="scope" />
```

  - Définition de la valeur d'une propriété d'un JavaBean ou d'un `Map` :

```
<c:set target="${nomObjet}" property="propriété" value="valeur" />
```

JSTL : `<c:set/>`

☞ Exemples :

- Définir une variable nommé **`user`**, initialisée avec la valeur du paramètre **`username`** et attaché à la request

```
<c:set var="user" value="${param.username}" scope="request"/>
```

- Réinitialiser la valeur de la propriété **`nom`** du JavaBean **`etudiant`**.

```
<c:set target="${etudiant}" property="nom" value="Moussa"/>
```

- Soit un **`Map<String, String>`** nommé **`map`** dans un scope. Ajouter une paire clé/valeur

```
<c:set target="${map}" property="DK4758-AA" value="Modou"/>
```

JSTL : <c:remove/>

- Le tag **<c:remove/>** permet de supprimer une variable attachée un scope (page, request, session et application).
- Syntaxe

```
<c:remove var="variable" scope="scope" />
```

- Remarque :
  Si aucun scope n'est précisé pour **<c:remove/>**, la variable est recherchée successivement dans **page**, **request**, **session** ou **application** et la première occurrence est supprimée.
- Exemple :

```
<c:remove var="user" scope="request"/>
```

JSTL : `<c:if/>`

- Le tag `<c:if/>` permet d'effectuer un traitement conditionnel comme l'instruction `if` en Java.
- Syntaxe

```
<c:if test="condition">
  code jsp ou html
</c:if>
```

- Le corps du tag est exécuté lorsque la valeur de l'attribut `test` est `true`.
- Exemple :

```
<c:if test="${langue eq 'fr'}">
  <jsp:forward page="indexFr.jsp"/>
</c:if>
```

JSTL `<c:choose>`

- Le tag `<c:choose>` permet de choisir parmis plusieurs instructions (jsp ou html) celle à exécuter en se basant sur la condition qui lui est associée.
- Le corps du tag `<c:choose>` est constitué d'un ou de plusieurs tags `<c:when>` et optionnellement d'un tag `<c:otherwise>`.
- `<c:choose>` effectue l'exécution du bloc conditionnel constitué du corps du tag `<c:when>` lorsque la condition de celui-ci est évaluée à `true`.
- Le tag `<c:otherwise>` (s'il est présent) est exécuté lorsqu'aucun tag `<c:when>` n'est exécuté.

JSTL <c:choose>

- Syntaxe

```
<c:choose>
  <c:when test="condition1"> code jsp ou html </c:when>
  ...
  <c:when test="condition2"> code jsp ou html </c:when>
  <c:otherwise> code jsp ou html </c:otherwise>
</c:choose>
```

- Exemple :

```
<c:choose>
    <c:when test="${langue eq 'fr'}">
        <jsp:forward page="indexFr.jsp"/>
    </c:when>
    <c:otherwise>
        <jsp:forward page="indexEn.jsp"/>
    </c:otherwise>
</c:choose>
```

JSTL `<c:forEach>`

- Le tag `<c:forEach>` permet d'effectuer un traitement itératif comme l'instruction `for` en Java.
- Syntaxe
  - Faire une boucle avec compteur

```
<c:forEach var="i" begin="iDebut" end="iFin">
  code jsp ou html
</c:forEach>
```

  - Parcourir une collection d'objets

```
<c:forEach var="element" items="collection" >
  code jsp ou html
</c:forEach>
```

JSTL `<c:forEach>`

- **`collection`** est une collection d'objet telle que :
  - tableau d'objets ou de primitifs
  - **`java.util.Map`**
  - **`java.util.Collection`**
  - **`java.util.Iterator`**
  - **`java.util.Enumeration`**

- Exemple 1 : Afficher les nombres impairs entre 1 et 10

```
<c:forEach var="i" begin="1" end="10" step="2">
    ${i} <br/>
</c:forEach>
```

JSTL `<c:forEach>`

☞ Exemples 2 :

- Parcourir un tableau et afficher ses éléments

```
<c:forEach var="elem" items="${tableau}">
    ${elem}<br/>
</c:forEach>
```

- Afficher les noms et valeurs des paramètres contenus dans la requête. L'objet implicite **`param`** est un **`Map`**.

```
<c:forEach var="p" items="${param}">
    ${p.key} = ${p.value}<br/>
</c:forEach>
```

JSTL <c:forTokens>

- Le tag <c:forTokens> permet de parcourir les sous-chaines d'une chaine de caractères séparées par des délimiteurs.
- Syntaxe

```
<c:forTokens var="elem" items="chaine" delims="delimiteurs">
    code jsp ou html
</c:forTokens>
```

- Exemple :

```
<c:forTokens var="e" items="windows,ubuntu-mac" delims=",-">
    ${e}<br/>
</c:forTokens>
```

JSTL `<c:url>`

- Le tag **`<c:url>`** permet de définir une url et :
  - d'ajouter **`JSESSIONID`** si les cookies sont désactivés au niveau du client (réécriture d'url).
  - d'ajouter des paramètres à l'url avec le sous-tag **`<c:param>`**
- Syntaxe

```
<c:url var="urlEncode" value="url">
    <c:param name="nom" value="valeur"/>
</c:url>
```

L'attribut **`var`** est optionnel, il permet de mettre l'url encodée dans la variable spécifiée.

- Exemple :

```
<c:url value="/erreur.jsp">
    <c:param name="periode" value="vérifier la période"/>
</c:url>
```

JSTL `<c:redirect>`

- Le tag `<c:redirect>` permet de demander au client de se rediriger vers l'emplacement spécifié par une url.
- `<c:redirect>` encode l'url comme le fait `<c:url>`.
- `<c:redirect>` peut utiliser le sous-tag `<c:param>` pour ajouter des paramètres à l'url.
- Syntaxe

```
<c:redirect url="url">
  <c:param name="nom" value="valeur"/>
</c:redirect>
```

- Exemple :

```
<c:redirect url="/ajoutMonnaie.jsp">
    <c:param name="nomMonnaie" value="valMonnaie"/>
</c:redirect>
```

JSTL `<c:import>`

- Le tag `<c:import>` permet d'importer la réponse d'une ressource interne ou externe à l'application web.
- La réponse de la ressource peut être insérée dans la réponse courante ou stockée dans une variable attachée à un scope.
- `<c:import>` peut utiliser le sous-tag `<c:param>` pour ajouter des paramètres de requête utiles pour la génération de la réponse de la ressource.

JSTL <c:import>

- Syntaxe

```
<c:import url="url" var="variable" scope="scope">
    <c:param name="nom" value="valeur"/>
</c:import>
```

  - L'attribut **var** est optionnel, il permet de contenir la réponse de la ressource dans la variable spécifiée.
  - L'attribut **scope** est optionnel, il permet de définir la portée de la variable.

- Exemple :

```
<c:import url="/menus.jsp" var="menus" scope="session">
    <c:param name="role" value="superuser"/>
</c:import>
```

JSTL `<c:out>`

- Le tag `<c:out>` évalue une expression et insère le résultat à la réponse.
- Si le résultat est `null` le tag `<c:out>` peut insérer une valeur par défaut.
- Syntaxe
  - le tag sans le corps

```
<c:out value="expression" escapeXml="true|false"
  default="expression par défaut" />
```

  - le tag avec le corps

```
<c:out value="expression" escapeXml="true|false">
  expression par défaut
</c:out>
```

JSTL `<c:out>`

- Attribut de `<c:out>`
  - `escapeXml` est optionnel, il indique si les caractères `<`, `>`, `&`, `'`, `"` doivent être remplacés par leurs codes respectifs : `<`, `>`, `&`, `'`, `"`.
  - `default` est optionnel, sa valeur par défaut est la chaine de caractères vide.
- Exemple : Afficher la photo de user s'il existe sinon la photo par défaut.

```
<img src="<c:out value="${user.photo}"
                 default="images/defaultPhoto.jpg"/>"
     alt="photo user"/>
```

JSTL `<c:catch>`

- Le tag `<c:catch>` permet de personnaliser la gestion des erreurs qui peuvent survenir lors du traitement d'une page jsp.
- Syntaxe

```
<c:catch var="refException">
    code pouvant générer une exception
</c:catch>
```

  - L'attibut `var` contient une référence à l'objet `Exception` généré

- Exemple

```
<c:catch var="erreur">
    <c:import url="menu.jsp"/>
</c:catch>

<c:if test="${not empty erreur}">
    La page <b>menu.jsp</b> est inaccessible !
    <p style="color:red;">${erreur.cause}</p>
</c:if>
```

JSTL : functions

- JSTL functions est la librairie pour la lecture de la taille d'une collection et la manipulation de chaine.
- Pour utiliser la librairie JSTL functions dans une page jsp, il suffit de la déclarer avec une directive **<%@ taglib %>**

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions"
    prefix="fn" %>
```

JSTL : functions

JSTL functions regroupe les fonctions

- **`length`** qui retourne le nombre d'éléments contenus dans une collection ou un tableau
  Exemple : Afficher le nombre de notes saisies par l'utilisateur

```
Nombre de notes = ${fn:length(paramValues.notes)}
```

- **`toUpperCase`** et **`toLowerCase`** pour modifier la casse d'une chaîne de caractère. Ces fonctions retournent chacune un **`String`**.
  Exemple : Mettre le nom de l'utilisateur en majuscule

```
${fn:toUpperCase(param.nom)}
```

JSTL : functions

- **`substring(chaine, iDebut, iFin)`**, **`substringBefore(chaine, sousChaine)`** et **`substringAfter(chaine, sousChaine)`** pour extraire une sous-chaine d'une chaine de caractères.

Exemple : Soit la chaine "Université Alioune Diop de Bambey".
Afficher la sous-chaine qui se trouve avant la sous-chaine "de Bambey".

```
${fn:substringBefore('Université Alioune Diop de Bambey',
      'de Bambey')}
```

JSTL : functions

- **`replace(chaine, sousChaine1, sousChaine2)`** pour remplacer une sous-chaine par une autre dans une chaine de caractères.

  Exemple : Remplacer " :" par "h" dans la chaine "12 :15"

```
${fn:replace('12:15', ':', 'h')}
```

- **`indexOf(chaine, sousChaine)`** retourne l'indice de la première occurrence de **`sousChaine`** dans la chaine.

  Exemple : Trouver l'indice du caractère " :" dans la chaine "12 :15"

```
${fn:indexOf('12:15', ':')}
```

JSTL : functions

- **startsWith**, **endsWith**, **contains** et **containsIgnoreCase** pour tester si une chaine de caractères contient sous-chaine spécifiée.

  Exemple : Afficher **oui** si la chaine "DatePeriodeIndex" contient la sous-chaine "Index"

```
<c:if test="${fn:contains('DatePeriodeIndex', 'Index')}">
   oui
</c:if>
```

- **escapeXml(chaine)** pour remplacer les caractères **<**, **>**, **&**, **'**, **"** contenus dans **chaine** par leurs codes respectifs : **<**, **>**, **&**, **'**, **"**.

JSTL : functions

- **`split(chaine, delim)`** pour subdiviser une chaine de caractères suivant un délimiteur. Cette fonction retourne un **`String[]`**.

  Exemple : Afficher les différents éléments de la chaine "modou :1234 :user :Mamadou"

```
<c:forEach var="e" items="${fn:split('modou:1234:user:Mamadou', ':')}">
   ${e}<br/>
</c:forEach>
```

- **`trim(chaine)`** pour supprimer les espaces situés au début et à la fin d'une chaine de caractères.

JSTL : functions

- **`join(tableau, delim)`** : pour créer une chaine de caractères à partir des éléments d'un tableau. La chaine créée contient les élements du tableau séparés par un délimiteur. Cette fonction retourne un **`String`**.

  Exemple : Lister les notes saisies par l'utilisateur (une note par ligne).

```
${fn:join(paramValues.notes, '<br/>')}
```